Experiment No. 01

Exploratory Data Analysis (EDA)

About the Topic

Exploratory Data Analysis is the first step of any data science project. Before building a model, we look at the data to understand its structure, spot missing values and outliers, and discover patterns and relationships between columns. Summary statistics such as mean, median and standard deviation, together with correlation checks, tell us which features look useful and which need cleaning.

What You Have to Perform

1. Load the dataset into a pandas DataFrame and display its shape, column types and first few rows.
2. Find missing values and handle them (drop or fill with mean, median or mode).
3. Compute summary statistics with describe() and count the categories in the categorical columns.
4. Calculate the correlation matrix and write 3-4 observations (for example, how survival differs by gender or class).

How to Implement

Open Google Colab, create a new notebook, and use pandas and numpy. Read the CSV directly from the dataset link with pd.read_csv(url), so no download or upload is needed.

Dataset

Titanic Passenger Dataset (CSV)

https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv

Observations / Result (to be filled by the student)

Experiment No. 02

Data Visualization - Charts, Graphs, Histogram, Scatter Plot, etc.

About the Topic

Data visualization turns raw numbers into pictures so that patterns become easy to see. A histogram shows how values of a single feature are distributed, a scatter plot shows the relationship between two features, a box plot highlights outliers, and a bar chart compares categories. Choosing the right chart for the right question is an essential skill for every data scientist.

What You Have to Perform

1. Load the Iris dataset and display the first rows.
2. Draw a histogram for each of the four flower measurements.
3. Draw a scatter plot of petal length versus petal width, coloured by species.
4. Draw a box plot and a bar chart (mean of each feature per species), and a pair plot of all features.
5. Add titles, axis labels and a legend to every chart and note what each chart tells you.

How to Implement

Use Google Colab with matplotlib and seaborn. The Iris dataset can be loaded directly using seaborn.load_dataset('iris') or sklearn.datasets.load_iris().

Dataset

Iris Flower Dataset (UCI Machine Learning Repository)
https://archive.ics.uci.edu/dataset/53/iris

Observations / Result (to be filled by the student)

Experiment No. 03

Linear Regression, Ridge and Lasso Regression

About the Topic

Linear regression predicts a continuous value (such as a house price) as a straight-line combination of input features by minimising the squared error. When there are many features, the model may overfit. Ridge regression adds an L2 penalty that shrinks the coefficients, while Lasso adds an L1 penalty that can shrink some coefficients exactly to zero, which also performs feature selection.

What You Have to Perform

1. Load the California Housing dataset and split it into training and test sets (80:20).
2. Train LinearRegression, Ridge and Lasso models using scikit-learn.
3. Calculate MSE and R-squared on the test set for all three models.
4. Change the regularization parameter alpha (for example 0.01, 0.1, 1, 10) and observe how the coefficients change.
5. Compare the three models in a small table and write a short conclusion.

How to Implement

Use Google Colab with scikit-learn, pandas and matplotlib. The dataset is built into scikit-learn and is downloaded automatically with fetch_california_housing().

Dataset

California Housing Dataset (scikit-learn)
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.fetch_california_housing.html

Observations / Result (to be filled by the student)

Experiment No. 04

Support Vector Machine (SVM) and k-Nearest Neighbours (kNN)

About the Topic

SVM is a classifier that finds the boundary (hyperplane) that separates two classes with the widest possible margin, and kernels allow it to handle non-linear data. kNN is a simple method that classifies a new point by looking at the majority class among its k closest training points. Both depend on distances, so feature scaling is important.

What You Have to Perform

1. Load the Breast Cancer Wisconsin dataset and split it into training and test sets.
2. Scale the features using StandardScaler.
3. Train a kNN classifier for k = 1, 3, 5, 7, 9 and plot accuracy against k.
4. Train SVM classifiers with linear and RBF kernels.
5. Compare the accuracy of kNN and SVM and print the confusion matrix of the best model.

How to Implement

Use Google Colab with scikit-learn. The dataset can be loaded using sklearn.datasets.load_breast_cancer(), so no file download is required.

Dataset

Breast Cancer Wisconsin (Diagnostic) Dataset (UCI)
https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

Observations / Result (to be filled by the student)

Experiment No. 05

Naive Bayes - Text Classification

About the Topic

Naive Bayes is a probabilistic classifier based on Bayes' theorem that assumes features are independent of each other. It works very well for text problems such as spam filtering. Text is first converted into numbers using Bag-of-Words or TF-IDF, and the classifier then estimates the probability of each class given the words in a message.

What You Have to Perform

1. Load the SMS Spam Collection dataset (labels: ham or spam).
2. Clean the text (lowercase, remove punctuation) and split it into training and test sets.
3. Convert the messages into vectors using CountVectorizer or TfidfVectorizer.
4. Train a MultinomialNB classifier and measure accuracy, precision and recall.
5. Test the model on 3-4 messages that you write yourself.

How to Implement

Use Google Colab with scikit-learn and pandas. Download the zip file from the link, upload the SMSSpamCollection file to Colab (or fetch it with wget), and read it with pd.read_csv(sep='\t').

Dataset

SMS Spam Collection Dataset (UCI)
https://archive.ics.uci.edu/dataset/228/sms-spam-collection

Observations / Result (to be filled by the student)

Experiment No. 06

Decision Tree and Random Forest

About the Topic

A decision tree classifies data by asking a series of yes/no questions on features, choosing at each step the split with the highest information gain (or lowest Gini impurity). Deep trees can overfit the training data. A Random Forest builds many trees on random samples of the data and features and combines their votes, which gives better and more stable accuracy.

What You Have to Perform

1. Load the Titanic dataset, select useful columns (class, sex, age, fare) and handle missing values.
2. Encode the categorical columns and split the data into training and test sets.
3. Train a DecisionTreeClassifier, visualise the tree with plot_tree, and try different max_depth values.
4. Train a RandomForestClassifier and compare its accuracy with the single tree.
5. Display the feature importances and identify the most important feature.

How to Implement

Use Google Colab with scikit-learn, pandas and matplotlib. Read the CSV directly from the dataset link using pd.read_csv(url).

Dataset

Titanic Passenger Dataset (CSV)
https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv

Observations / Result (to be filled by the student)

Experiment No. 07

Feature Engineering

About the Topic

Feature engineering is the process of creating, transforming and selecting input features so that a model can learn better. It includes handling missing values, encoding categorical variables, scaling numerical values, and creating new features from existing ones. Good features often improve model performance more than choosing a more complicated algorithm.

What You Have to Perform

1. Load the Titanic dataset and fill the missing values in age and embarked.
2. Create new features such as family_size (sibsp + parch + 1), is_alone, and age groups (child, adult, senior).
3. Apply one-hot encoding to categorical columns and scale age and fare with StandardScaler or MinMaxScaler.
4. Select the best features using SelectKBest or feature importances.
5. Train a simple model before and after feature engineering and compare the accuracy.

How to Implement

Use Google Colab with pandas and scikit-learn. Continue with the same Titanic CSV used in Experiments 1 and 6, loaded via pd.read_csv(url).

Dataset

Titanic Passenger Dataset (CSV)

https://raw.githubusercontent.com/mwaskom/seaborn-data/master/titanic.csv

Observations / Result (to be filled by the student)

Experiment No. 08

Principal Component Analysis (PCA)

About the Topic

PCA is a dimensionality reduction technique that converts many correlated features into a smaller number of new, uncorrelated features called principal components. The first component captures the largest possible variance in the data, the second captures the next largest, and so on. PCA is used for visualisation, noise reduction and speeding up models.

What You Have to Perform

1. Load the Wine dataset (13 features) and standardise the features.
2. Apply PCA and print the explained variance ratio of each component.
3. Plot the cumulative explained variance and decide how many components keep about 95% of the information.
4. Plot the data on the first two principal components, coloured by wine class.
5. Train a classifier on the original data and on the PCA data and compare accuracy.

How to Implement

Use Google Colab with scikit-learn and matplotlib. The dataset is available using sklearn.datasets.load_wine(), so no download is needed.

Dataset

Wine Dataset (UCI)

https://archive.ics.uci.edu/dataset/109/wine

Observations / Result (to be filled by the student)

Experiment No. 09

Neural Network

About the Topic

An artificial neural network is made of layers of connected neurons. Each neuron takes a weighted sum of its inputs and passes it through an activation function such as ReLU or softmax. The network learns by adjusting its weights through backpropagation and gradient descent so that the loss decreases over many epochs.

What You Have to Perform

1. Load the MNIST handwritten digits dataset and normalise the pixel values to the range 0-1.
2. Build a simple network with a Flatten layer, one or two Dense hidden layers (ReLU) and a Dense output layer of 10 units (softmax).
3. Compile the model (Adam optimizer, sparse categorical cross-entropy) and train it for 5-10 epochs.
4. Plot the training and validation accuracy and loss curves.
5. Evaluate on the test set and display a few predictions along with the actual digits.

How to Implement

Use Google Colab with TensorFlow/Keras. MNIST is built into Keras and loads with keras.datasets.mnist.load_data(). Enable a GPU from Runtime > Change runtime type if you want faster training.

Dataset

MNIST Handwritten Digits Dataset (Keras)
https://keras.io/api/datasets/mnist/

Observations / Result (to be filled by the student)

Experiment No. 10

Image Classification using CNN

About the Topic

A Convolutional Neural Network (CNN) is designed for image data. Convolution layers apply small filters that detect edges, textures and shapes, pooling layers reduce the image size, and fully connected layers make the final decision. Because filters are shared across the image, CNNs need far fewer parameters than ordinary networks and perform much better on images.

What You Have to Perform

1. Load the CIFAR-10 dataset (10 classes of 32x32 colour images) and normalise the images.
2. Build a CNN with 2-3 Conv2D + MaxPooling2D blocks, followed by Flatten and Dense layers.
3. Train the model for 10 epochs and plot accuracy and loss.
4. Evaluate on the test set and show a few test images with predicted and actual labels.
5. Optional: add Dropout or data augmentation and observe the change in accuracy.

How to Implement

Use Google Colab with TensorFlow/Keras and switch to a GPU runtime. CIFAR-10 loads directly with keras.datasets.cifar10.load_data().

Dataset

CIFAR-10 Image Dataset (Keras)
https://keras.io/api/datasets/cifar10/

Observations / Result (to be filled by the student)

Experiment No. 11

Analysis and Implementation of Various Performance Metrics

About the Topic

Accuracy alone does not tell the whole story of a classifier, especially when classes are imbalanced. A confusion matrix counts true and false positives and negatives, from which precision, recall and F1-score are calculated. The ROC curve and AUC show how well the model separates classes at different thresholds. For regression, MAE, MSE, RMSE and R-squared are used.

What You Have to Perform

1. Load the Breast Cancer dataset and train a Logistic Regression model and a Decision Tree.
2. Compute and plot the confusion matrix of each model.
3. Calculate accuracy, precision, recall and F1-score using scikit-learn, and also compute them manually from the confusion matrix.
4. Plot the ROC curves of both models on one graph and compare their AUC values.
5. Conclude which model is better and explain why one metric may matter more than another here.

How to Implement

Use Google Colab with scikit-learn and matplotlib. The dataset loads with sklearn.datasets.load_breast_cancer().

Dataset

Breast Cancer Wisconsin (Diagnostic) Dataset (UCI)
https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic

Observations / Result (to be filled by the student)

Experiment No. 12

k-Means Clustering

About the Topic

k-Means is an unsupervised algorithm that groups unlabelled data into k clusters. It places k centroids, assigns every point to its nearest centroid, moves each centroid to the mean of its points, and repeats until the clusters stop changing. The right value of k can be chosen using the elbow method or the silhouette score.

What You Have to Perform

1. Load the Iris dataset and use only the four numerical features (ignore the species labels).
2. Run k-Means for k = 1 to 10 and plot the elbow curve (inertia versus k).
3. Choose a suitable k, fit the model and compute the silhouette score.
4. Plot the clusters and centroids using two features (or the first two PCA components).
5. Compare the clusters with the actual species and comment on how well they match.

How to Implement

Use Google Colab with scikit-learn and matplotlib. The dataset loads with sklearn.datasets.load_iris() or seaborn.load_dataset('iris').

Dataset

Iris Flower Dataset (UCI Machine Learning Repository)
https://archive.ics.uci.edu/dataset/53/iris

Observations / Result (to be filled by the student)

Experiment No. 13

Hierarchical Clustering

About the Topic

Hierarchical clustering builds a tree of clusters called a dendrogram. In the agglomerative approach, every point starts as its own cluster and the two closest clusters are merged repeatedly until only one remains. The linkage method (single, complete, average or Ward) decides how the distance between clusters is measured, and cutting the dendrogram at a chosen height gives the final clusters.

What You Have to Perform

1. Load the Penguins dataset, drop rows with missing values and select the numerical columns.
2. Standardise the features using StandardScaler.
3. Draw dendrograms using scipy with different linkage methods (single, complete, ward).
4. Apply AgglomerativeClustering with a suitable number of clusters.
5. Compare the clusters with the actual penguin species and note which linkage works best.

How to Implement

Use Google Colab with scipy, scikit-learn and seaborn. Read the CSV directly from the link with pd.read_csv(url).

Dataset

Palmer Penguins Dataset (CSV)
https://raw.githubusercontent.com/mwaskom/seaborn-data/master/penguins.csv

Observations / Result (to be filled by the student)

Experiment No. 14

NoSQL using MongoDB

About the Topic

MongoDB is a popular NoSQL document database that stores data as flexible JSON-like documents inside collections, instead of rows and tables. It does not need a fixed schema and scales easily for big data. Basic operations are CRUD: insert, find (with filters, projection and sorting), update and delete, plus aggregation pipelines for grouping and summarising data.

What You Have to Perform

1. Create a free MongoDB Atlas cluster (or use a local MongoDB) and connect using Python's pymongo.
2. Load the sample dataset (for example, the sample_mflix movies collection) or insert documents from a CSV/JSON file.
3. Perform CRUD operations: insert a few documents, find with a filter and sort, update a field, and delete a document.
4. Run an aggregation query, such as the number of movies per genre or the average rating per year.
5. Print the results and write a short note on how this differs from an SQL table.

How to Implement

Use Google Colab with pip install pymongo, and connect to a free MongoDB Atlas (M0) cluster using the connection string. Alternatively, use MongoDB Compass or mongosh on your own system.

Dataset

MongoDB Atlas Sample Datasets
https://www.mongodb.com/docs/atlas/sample-data/

Observations / Result (to be filled by the student)